# [code_analysis] Parser - Lexing and Parsing SQL

| | |
|---|---|
| Created By | Hyung-Gyu Ryoo |
| Email | hgryoo@cubrid.com |
| Tags | 코드분석 |
| Team | 개발2팀 |
| Version | 1.0 |


CUBRID Parser - Lexing and Parsing SQL
- 어휘 분석과 구문 분석
- 어휘 분석 규칙을 만들기 위한 csql_lexer.l 파일
- 구문 분석 규칙을 만들기 위한 csql_grammar.y 파일
- csql_lexer.l과 csql_grammar.y가 빌드되는 과정


# CUBRID Parser - Lexing and Parsing SQL

## 어휘 분석과 구문 분석

큐브리드에서는 Flex와 Bison 도구를 사용하여 입력된 SQL 문자열에 대해 각각 어휘 분석과 구문 분석 (parsing) 을 수행한다. 어휘 분석이란 SQL 구문을 정규 문법에 따라 나누어 질 수 없는 토큰 (token)으로 구분하는 것이다. 그리고 구문 분석에서 매칭되는 토큰의 패턴에 따라 지정된 동작을 수행한다. 구문 분석을 수행하고 나면 **최종적으로 입력된 SQL 구문에 대응하는 PT_NODE로 이루어진 트리 구조가 생성된다.**

## 어휘 분석 규칙을 만들기 위한 csql_lexer.l 파일

csql_lexer.l 파일에서 Flex를 사용한 어휘 분석을 수행하기 위한 룰을 작성한다. 이 파일은 크게 세 부분으로 구분된다.

```
Lexer 파일 구조
===========
%{
Define section
%}

%%
Rules section
%%

User subroutine
```

- **Define section:** 다른 헤더파일을 가져오거나 변수, 함수의 정의 등을 포함한다. 다음은 큐브리드 코드의 예시이다. 파서 노드 관련한 정의나 시스템 파라미터 그리고 에러 시 에러 코드를 반환하기 위한 메시지 카탈로그 등의 헤더파일을 가져온다.

```
%{
#include "csql_grammar.h"
#include "csql_grammar_scan.h"
#include "parse_tree.h"
#include "parser_message.h"
#include "system_parameter.h"
#include "message_catalog.h"

#if defined (SUPPRESS_STRLEN_WARNING)
#define strlen(s1)  ((int) strlen(s1))
#endif /* defined (SUPPRESS_STRLEN_WARNING) */

#define CSQL_MAXNAME  256

static int parser_yyinput_single_line (char *buff, int max_size);
static int parser_yyinput_multi_line (char *buff, int max_size);
static int parser_yyinput (char *buff, int max_size);
```

```
static char *parser_c_hint (void);
static char *parser_line_hint (void);
static void begin_token (char *token);

extern int yyline;
extern int yycolumn;
extern int yycolumn_end;
extern int dot_flag;

int str_identifier = '\0';

#undef YY_INPUT
#define YY_INPUT(buffer, result, max_size) { result = parser_yyinput(buffer, max_size); result == 0 ? result = YY_NULL : re

/*
 * This action is executed each time a token is matched. Its position in the
 * buffer is saved as part of the location information.
 */
#define YY_USER_ACTION  {yybuffer_pos += csql_yyget_leng (); csql_yylloc.buffer_pos = yybuffer_pos;}

#define IS_BLANK_CHAR(c) \
                    ((c) == ' ' || (c) == '\t' || (c) == '\r' || (c) == '\n')

int g_msg[1024];
int msg_ptr;

int yybuffer_pos;
%}
```

- **Rules section:** 이 섹션에서는 각 토큰에 대한 어휘규칙과 동작 코드를 작성한다. **큐브리드에서 사용할 새로운 토큰을 추가한다면 대부분 이 섹션의 내용을 참고하면 된다.** 작성 규칙은 `어휘규칙 공백 C코드` 형식으로 작성한다. 다음에서 예시로 큐브리드 코드의 일부분을 가져왔다.

```
%%
...
[sS][eE][lL][eE][cC][tT]            { begin_token(yytext);   return SELECT; }
[sS][eE][nN][sS][iI][tT][iI][vV][eE]          { begin_token(yytext);   return SENSITIVE; }
[sS][eE][pP][aA][rR][aA][tT][oO][rR]          { begin_token(yytext);
                    csql_yylval.cptr = pt_makename(yytext);
                    return SEPARATOR; }
...
%%
```

- **User subroutine section:** 이 부분에는 보통 규칙에서 호출하는 루틴을 담고 있다. yacc에서 사용할 여러가지 함수를 작성하고, yacc에서 기본적으로 사용하는 함수들인 input(), unput(), output(), ywrap()을 재정의할 수 있는데, 이 구역에 재정의하여 구현하면 된다. 큐브리드에서는 다음의 함수들을 구현하고 있다.

```
/* 에러를 위한 별도 루틴 */
csql_yyerror_explicit (int line, int column)
csql_yyerror (const char *s)

/*
* yywrap은 입력의 끝을 만났을 때의 처리를 정의하는데,
*  그냥 1을 반환하도록 재정의하고 있다.
*/
yywrap ()

/* cubrid custom input functions */
parser_yyinput (char *buff, int max_size)
parser_yyinput_single_line (char *buff, int max_size)
parser_yyinput_multi_line (char *buff, int max_size)

input_internal () // 내부에서 input()을 호출한다 line과 column 정보를 할당한다
parser_c_hint (void)
parser_line_hint (void)
begin_token (char *token) // 각 토큰이 시작할 때 호출한다
```

## 구문 분석 규칙을 만들기 위한 csql_grammar.y 파일

csql_grammar.y 파일은 Bison에서 사용된다. Bison 문법의 구조에 따라 다음 4개의 섹션으로 나뉜다.

```
%{
  Prologue
%}

Bison declarations

%%
Grammar rules
%%

Epilogue
```

- **Prologue**: 이 섹션에서는 함수와 매크로, 그리고 Grammar rules 섹션에서 사용할 변수를 정의한다. 또 #include 구문으로 다른 위치의 헤더파일을 로드할 수 있다. 큐브리드에서는 구문 분석 결과로 만들어져야 하는 PT_NODE의 정의를 담고 있는 파일을 포함시키거나 구문 분석에 필요한 여러 가지 변수 및 구조체를 정의하고 있다.

```
%{/*%CODE_REQUIRES_START%*/
#include "json_table_def.h"
#include "parser.h"

/*
 * The default YYLTYPE structure is extended so that locations can hold
 * context information
 */
typedef struct YYLTYPE
{

  int first_line;
  int first_column;
  int last_line;
  int last_column;
  int buffer_pos; /* position in the buffer being parsed */

} YYLTYPE;
... (생략)
%}
```

- **Bison declarations:** terminal, non-terminal 심볼에 대한 정의를 포함한다. 예를 들어 구문 분석 중 정의한 심볼을 만나면 각 심볼의 결과로 만들어 질 데이터 타입을 **%type**의 꺽쇠<>에 입력한다. 이 때 꺽쇠 안에는 **%union**에 정의한 것들 중 하나를 적는다. 그 예시는 다음과 같다.

```
%union
{
  int number;
  bool boolean;
  PT_NODE *node;
  char *cptr;
  container_2 c2;
  container_3 c3;
  container_4 c4;
  container_10 c10;
  struct json_table_column_behavior jtcb;
}

%type <number> show_type
%type <node> stmt
... (생략)
```

- **Grammar rules:** 이 섹션에서는 구문 분석을 위한 문법 규칙을 정의한다. **새로운 문법을 정의하거나, SQL을 파싱하는 방법을 바꾼다면 이 부분을 참고한다.**

  문법 규칙을 정의하는 방법은 다음과 같다.

  `result : components ... ;`

  이 때 ...에서 이 문법 규칙을 부합하면 수행할 action을 정의할 수 있다. 수행할 action이 없다면 생략하는 것도 가능하다.

  `{ C statements }`

이를 기반으로 큐브리드에서 정의하는 문법 규칙 중 일부로, WITH 절을 포함하는 SELECT 문에 대한 규칙을 예를 들어 보여준다면 다음과 같다. 부합하는 component는 `$`를 이용하여 `$1, $2`와 같이 action 블록 내에서 변수로 assign할 수 있다.

```
%type <node> select_expression_opt_with

%%
... (생략)
select_expression_opt_with
: opt_with_clause
  select_expression
  {{

    PT_NODE *with_clause = $1; // opt_with_clause에 대한 component
    PT_NODE *stmt = $2; // select_expression에 대한 component
    if (stmt && with_clause)
      {
        stmt->info.query.with = with_clause;
      }

    $$ = stmt;

  DBG_PRINT}}
;
... (생략)
%%
```

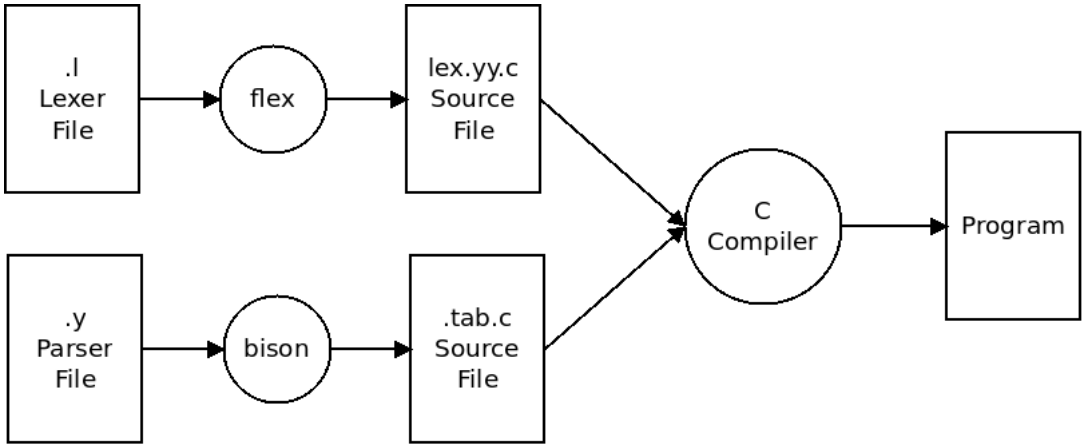
- **Epilogue:** yylex와 yyerror와 같이 이미 정의된 함수를 재정의할 수 있다. 보통 Bison parser는 yy나 YY로 시작하는 매크로 또는 식별자를 가진, 미리 정의되어 있는 함수들을 가진다. 이를 C-language interface라 한다.
  큐브리드에서는 대부분 파싱 과정에서 PT_NODE를 만들거나 함수들로 정의되어 있다. 그 예시는 다음과 같다.

```
/* argument 갯수를 입력으로 PT_FUNCTION 노드를 만듬 */
PT_NODE *
parser_make_func_with_arg_count (PARSER_CONTEXT * parser, FUNC_TYPE func_code, PT_NODE * args_list,
                                 size_t min_args, size_t max_args)

/* Operator 타입과 인수를 입력으로 PT_EXPR 노드를 만듬 */
PT_NODE *
parser_make_expression (PARSER_CONTEXT * parser, PT_OP_TYPE OP, PT_NODE * arg1, PT_NODE * arg2,
      PT_NODE * arg3)

/* next로 노드를 연결한다 */
PT_NODE *
parser_make_link (PT_NODE * list, PT_NODE * node)
{
  parser_append_node (node, list);
  return list;
}
```

## csql_lexer.l과 csql_grammar.y가 빌드되는 과정



각각의 파일은 위 그림과 같이 flex와 bison 도구에 의해 전처리 과정을 거치고 나면 c 파일이 생성된다. 이렇게 만들어진 c 파일을 포함하여 큐브리드 전체에 대한 빌드를 수행한다.

다음은 큐브리드 소스코드의 최상위 폴더에 있는 CMakeLists.txt 파일의 일부이다. 이 빌드 커맨드 부분에서 Flex와 Bison을 어떻게 사용하여 빌드하는지 참고할 수 있다. csql_lexer.l과 csql_grammar.y를 수정하면 csql_grammar.yy가 생성되고, 이를 빌드 target에 추가한다.

```
# CSQL FLEX/BISON targets
# replace old bison directives with new ones
# use add_custom_command because we want to regenerate csql_grammar.yy
# whenever csql_grammar.y is modified
add_custom_command(OUTPUT ${CMAKE_CURRENT_BINARY_DIR}/csql_grammar.yy
  COMMAND ${CMAKE_COMMAND} -DGRAMMAR_INPUT_FILE=${CMAKE_SOURCE_DIR}/src/parser/csql_grammar.y -DGRAMMAR_OUTPUT_FILE=${CMAKE_CUR
  MAIN_DEPENDENCY ${CMAKE_SOURCE_DIR}/src/parser/csql_grammar.y
)

set(CSQL_GRAMMAR_INPUT ${CMAKE_CURRENT_BINARY_DIR}/csql_grammar.yy)

# because csql_grammar.yy are generated through a custom command
# whenever csql_grammar.y is modified, bison and flex will be
# run to create new outputs

bison_target(csql_grammar ${CSQL_GRAMMAR_INPUT} ${CSQL_GRAMMAR_OUTPUT}
  COMPILE_FLAGS "--no-lines --name-prefix=csql_yy -d -r all")
flex_target(csql_lexer ${CSQL_LEXER_INPUT} ${CSQL_LEXER_OUTPUT}
  COMPILE_FLAGS "--noline --never-interactive --prefix=csql_yy")
add_flex_bison_dependency(csql_lexer csql_grammar)
add_custom_target(gen_csql_grammar DEPENDS ${BISON_csql_grammar_OUTPUTS})
add_custom_target(gen_csql_lexer DEPENDS ${FLEX_csql_lexer_OUTPUTS})
```